

Implementation Work Division and Deliverables Checklist

Project Context

This document divides the implementation work of the project **DeepFake Video Detection System for Video KYC in Financial Systems Using rPPG and Quantum Machine Learning on Low-Resolution Videos** among three team members: Aswin, Sumit, and Jhashank. The work breakdown follows a deliverable-oriented machine learning pipeline, where large system modules are split into smaller work packages so that multiple team members can work in parallel instead of waiting for one linear sequence to finish [web:255][web:257][web:261].

The system itself follows a typical deepfake detection structure with input handling, preprocessing, feature extraction, classification, and evaluation, which makes it practical to assign ownership module by module [web:256][web:259]. Since the project is based on a two-stage architecture where **rPPG is the primary evidence layer** and **quantum machine learning is the final decision layer**, the task split below is organized around those technical blocks [web:213][web:219].

Team Roles

Member	Role Style	Main Strength Area	Implementation Position
Aswin	Research-oriented	Planning, experiments, validation, documentation	Preprocessing control, experiment pipeline, integration tracking
Sumit	Technical build	Data pipeline, preprocessing, face pipeline	Input-to-ROI pipeline owner
Jhashank	Technical build	Model coding, feature extraction, training	rPPG-to-classifier pipeline owner

The division is designed so that all three members contribute to implementation, but the coding load is not identical because responsibilities are matched to skill profile and project needs [web:255][web:261]. This also allows all three to work at the same time on different deliverables under the same architecture [web:257][web:258].

Working Rule

Each architecture block below contains three things:

- **Owner:** person mainly responsible.
- **Support:** person who helps with checking, testing, or integration.
- **Deliverables checklist:** concrete tasks that can be ticked.

A task is considered complete only when its code, output, folder structure, and validation notes are all available in the shared project workspace [web:255][web:258].

1. Input Acquisition Layer

Objective: Accept FaceForensics++ videos, organize them correctly, and prepare them for frame-level processing [web:256][web:259].

Primary Owner: Sumit

Support: Aswin

Deliverables

- ☐ Create master dataset folder structure for real and fake video classes.
- ☐ Separate training, validation, and test folders or split files.
- ☐ Verify FaceForensics++ file paths, naming format, and video readability.
- ☐ Create script to scan all videos and report corrupt/unreadable files.
- ☐ Record video metadata: resolution, fps, duration, file size.
- ☐ Prepare dataset inventory CSV or sheet with video IDs and labels.
- ☐ Share dataset structure with full team.

Aswin support tasks

- ☐ Define split strategy and naming convention document.
- ☐ Verify that the class distribution is balanced enough for experiments.
- ☐ Prepare checklist for what counts as a usable or unusable video.

2. Frame Sampling Layer

Objective: Convert videos into usable frame sequences at a controlled sampling rate so that later modules work on consistent temporal data [web:256][web:259].

Primary Owner: Sumit

Support: Jhashank

Deliverables

- ☐ Create video-to-frame extraction script.
- ☐ Decide and implement frame sampling rate.
- ☐ Save frames in ordered sequence for every video.
- ☐ Ensure frame names preserve time order.
- ☐ Create logs for extraction success/failure.
- ☐ Test extraction on sample real and fake videos.
- ☐ Store extracted frame counts per video.

Jhashank support tasks

- ☐ Confirm sampled sequence length is sufficient for temporal modeling.
- ☐ Check whether extracted frames work with planned rPPG module input.

Aswin parallel tasks

- ☐ Make a comparison note for different frame sampling rates.
- ☐ Finalize one project-standard sampling setting for baseline experiments.

3. Frame Quality Assessment Layer

Objective: Identify low-quality or unusable frames before they damage rPPG and classification quality [web:222][web:226].

Primary Owner: Aswin

Support: Sumit

Deliverables

- ☐ Define blur threshold rule.
- ☐ Define brightness threshold rule.
- ☐ Define face visibility rule.
- ☐ Define frame rejection categories such as blur, dark frame, no face, extreme pose.
- ☐ Create frame-quality checklist sheet.
- ☐ Test quality rules on sample outputs.
- ☐ Mark example good and bad frames for reporting.

Sumit support tasks

- ☐ Add frame-quality scoring to extraction pipeline if possible.
- ☐ Export quality flags along with frame metadata.

Why Aswin owns this

This is a real implementation task but lighter in advanced model coding, and it directly supports preprocessing design and experimental rigor [web:255][web:261].

4. Face Detection Layer

Objective: Detect the face in each frame so the system focuses only on the relevant subject region [web:256][web:259].

Primary Owner: Sumit

Support: Jhashank

Deliverables

- ☐ Select face detection library or model.
- ☐ Run face detection on all sampled frames.
- ☐ Save bounding box coordinates.
- ☐ Create fallback rule for missed detections.
- ☐ Reject or flag frames where no proper face is found.
- ☐ Measure face detection success rate.
- ☐ Save visual examples of correct and incorrect detections.

Support tasks

- ☐ Jhashank checks whether cropped face format matches model input expectation.
- ☐ Aswin reviews random outputs and notes common failure cases.

5. Face Tracking and Alignment Layer

Objective: Stabilize the face across frames so the same facial regions stay consistent over time, which is critical for rPPG [web:213][web:216].

Primary Owner: Sumit

Support: Aswin

Deliverables

- ☐ Implement face crop generation from detected boxes.
- ☐ Add alignment using facial landmarks if feasible.
- ☐ Smooth face crop position across adjacent frames.
- ☐ Maintain consistent crop size across the sequence.
- ☐ Save aligned face sequence folders.
- ☐ Create sample before/after alignment visualization.
- ☐ Document alignment failures.

Aswin support tasks

- ☐ Check whether face stabilization improves ROI consistency.
- ☐ Prepare qualitative notes for report screenshots.

6. Low-Resolution Preprocessing Layer

Objective: Standardize low-res videos without damaging the weak forensic and physiological clues [web:222][web:239].

Primary Owner: Aswin

Support: Sumit

Deliverables

- ☐ Decide final input size for face crops.
- ☐ Create resized low-resolution face dataset.
- ☐ Define brightness normalization rule.
- ☐ Define optional mild denoising rule.
- ☐ Decide whether color normalization is needed.
- ☐ Create preprocessing configuration sheet.
- ☐ Prepare comparison set: raw low-res vs normalized low-res.
- ☐ Document reason for not using aggressive super-resolution in baseline system.

Sumit support tasks

- ☐ Integrate resizing and normalization into preprocessing script.
- ☐ Save processed outputs in architecture-compliant folder structure.

Jhashank support tasks

- ☐ Verify processed data remains compatible with rPPG extraction.

7. ROI Selection for rPPG Layer

Objective: Isolate skin-rich facial regions such as cheeks and forehead for physiological signal extraction [web:213][web:216].

Primary Owner: Jhashank

Support: Sumit

Deliverables

- ☐ Decide ROI strategy: left cheek, right cheek, forehead, or combined mask.
- ☐ Implement ROI extraction from aligned faces.
- ☐ Maintain ROI consistency across frame sequence.
- ☐ Save ROI preview images for sample videos.
- ☐ Create log of ROI failures due to pose or occlusion.
- ☐ Verify ROI coverage does not include too much background.

Support tasks

- ☐ Sumit helps with landmark-based region localization if needed.
- ☐ Aswin checks whether ROI outputs are report-ready and interpretable.

8. rPPG Signal Extraction Layer

Objective: Extract pulse-related temporal signals from facial ROIs because rPPG is the main hero module of the project [web:213][web:216].

Primary Owner: Jhashank

Support: Aswin

Deliverables

- ☐ Build temporal signal extraction from ROI color traces.
- ☐ Aggregate signal across selected frames.
- ☐ Apply normalization or signal cleaning.
- ☐ Store signal sequence for each video.
- ☐ Generate sample plots of extracted signals.
- ☐ Compare signal quality between real and fake samples.
- ☐ Save signal features in structured format for later classification.

Aswin support tasks

- ☐ Prepare experiment sheet for signal comparison.
- ☐ Document which signals appear stable, noisy, or suspicious.

9. rPPG Feature Engineering Layer

Objective: Convert raw temporal signals into useful features for classification [web:216][web:226].

Primary Owner: Jhashank

Support: Aswin

Deliverables

- ☐ Define feature list, such as temporal consistency, inter-region agreement, stability score, amplitude score, rhythm quality.
- ☐ Build feature extraction script.
- ☐ Create feature vector for each video.
- ☐ Save feature matrix for training and testing.
- ☐ Validate that features are not empty or corrupted.
- ☐ Compare feature distributions for real vs fake classes.

Aswin support tasks

- ☐ Maintain feature documentation table.
- ☐ Decide which features must be included in first baseline.

10. Quantum Input Conditioning Layer

Objective: Prepare compact features so they can be used by the hybrid quantum decision module [web:219][web:224].

Primary Owner: Jhashank

Support: Aswin

Deliverables

- ☐ Normalize feature vectors.
- ☐ Reduce or select dimensions if required.
- ☐ Create final input tensor/vector format for quantum stage.
- ☐ Validate shape consistency across all samples.
- ☐ Save train/val/test feature files.
- ☐ Document feature-to-quantum mapping approach.

Aswin support tasks

- ☐ Write simple explanation of why compact features are used before quantum stage.
- ☐ Track experiment variants with different feature sets.

11. Quantum Machine Learning Layer

Objective: Build the final hybrid classification block that takes conditioned features and predicts real or fake [web:219][web:224].

Primary Owner: Jhashank

Support: Aswin

Deliverables

- ☐ Select quantum ML framework or library.
- ☐ Create baseline hybrid quantum classifier.
- ☐ Connect conditioned feature input to classifier.
- ☐ Train the model on the prepared feature set.
- ☐ Save model checkpoints and training logs.
- ☐ Record train, validation, and test performance.
- ☐ Compare quantum classifier against one classical baseline.

Aswin support tasks

- ☐ Maintain experiment tracking sheet.
- ☐ Prepare results table for paper and review meetings.
- ☐ Write plain-language interpretation of model behavior.

12. Output Decision Layer

Objective: Convert model outputs into practical KYC-friendly classes such as real, fake, and uncertain [web:214][web:217].

Primary Owner: Aswin

Support: Jhashank

Deliverables

- ☐ Define final decision categories.
- ☐ Define confidence thresholds.
- ☐ Create result summary format for each video.
- ☐ Decide how uncertain cases will be marked.
- ☐ Create output template for screenshots and reporting.
- ☐ Validate that outputs are understandable to non-technical reviewers.

Jhashank support tasks

- ☐ Export prediction scores in usable format.
- ☐ Help calibrate thresholds using validation results.

13. Evaluation Layer

Objective: Measure how well the full system works using proper machine learning evaluation outputs [web:255][web:261].

Primary Owner: Aswin

Support: Jhashank

Deliverables

- ☐ Define evaluation metrics: accuracy, precision, recall, F1-score, confusion matrix.
- ☐ Define low-resolution evaluation setup.
- ☐ Build results comparison table for all experiment versions.
- ☐ Record baseline and final system results.
- ☐ Prepare error analysis sheet.
- ☐ Identify common failure cases.

- ☐ Prepare graphs/tables needed for report.

Jhashank support tasks

- ☐ Generate predictions and metric-ready outputs.
- ☐ Help in ablation comparisons.

Sumit support tasks

- ☐ Help connect failed cases back to preprocessing or face-detection issues.

14. Integration Layer

Objective: Join all modules into one end-to-end working pipeline [web:255][web:258].

Primary Owner: Aswin

Support: Sumit and Jhashank

Deliverables

- ☐ Create architecture-level checklist for every module input/output.
- ☐ Verify folder compatibility between Sumit and Jhashank modules.
- ☐ Create end-to-end run order document.
- ☐ Prepare demo pipeline on a small subset.
- ☐ Track bugs found in integrated run.
- ☐ Maintain version control notes or shared progress log.
- ☐ Confirm final pipeline follows project architecture.

Sumit tasks in integration

- ☐ Fix data path and preprocessing compatibility issues.
- ☐ Ensure face/ROI outputs are correctly handed over.

Jhashank tasks in integration

- ☐ Fix model input mismatches.
- ☐ Ensure output scores are exported correctly.

Ordered Execution Plan

The project must look parallel, so the ordered execution should be understood as **priority order inside each person's track**, not a strict one-after-another sequence [web:257][web:261].

Aswin track

1. ☐ Define dataset split plan.
2. ☐ Define frame quality rules.
3. ☐ Define low-resolution preprocessing policy.
4. ☐ Create preprocessing checklist.
5. ☐ Create experiment tracking sheet.
6. ☐ Define output labels and thresholds.
7. ☐ Build evaluation tables.
8. ☐ Track integration and results.

Sumit track

1. ☐ Organize dataset.
2. ☐ Build video-to-frame extraction.
3. ☐ Add frame quality export support.
4. ☐ Implement face detection.
5. ☐ Implement face cropping.
6. ☐ Implement alignment and smoothing.
7. ☐ Integrate low-resolution preprocessing.
8. ☐ Hand over ROI-ready facial sequences.

Jhashank track

1. ☐ Confirm input format for modeled sequence.
2. ☐ Implement ROI extraction.
3. ☐ Implement rPPG signal extraction.
4. ☐ Build rPPG features.
5. ☐ Prepare conditioned feature vectors.
6. ☐ Build hybrid quantum classifier.
7. ☐ Train and validate model.
8. ☐ Export prediction scores and logs.

Parallel Milestone Plan

Milestone 1: Setup Week

- ☐ Sumit finishes dataset structure and frame extraction starter.
- ☐ Jhashank finalizes expected input format for rPPG pipeline.
- ☐ Aswin finalizes split rules, quality rules, and preprocessing sheet.

Milestone 2: Preprocessing Week

- ☐ Sumit completes face detection and aligned face crops.
- ☐ Aswin completes low-resolution normalization policy and quality validation.
- ☐ Jhashank starts ROI and temporal signal extraction.

Milestone 3: Feature Week

- ☐ Jhashank completes rPPG feature generation.
- ☐ Aswin creates feature documentation and evaluation table structure.
- ☐ Sumit supports debugging of ROI and face consistency issues.

Milestone 4: Model Week

- ☐ Jhashank completes baseline quantum/classical model runs.
- ☐ Aswin records experiment results and threshold settings.
- ☐ Sumit supports data corrections if training issues come from preprocessing.

Milestone 5: Integration Week

- ☐ All modules run together on a common subset.
- ☐ Prediction outputs are generated.
- ☐ Evaluation table is finalized.
- ☐ Error cases are documented.

Final Ownership Summary

Architecture Block	Main Owner	Secondary Support
Input acquisition	Sumit	Aswin
Frame sampling	Sumit	Jhashank
Frame quality	Aswin	Sumit
Face detection	Sumit	Jhashank
Face alignment	Sumit	Aswin
Low-resolution preprocessing	Aswin	Sumit
ROI extraction	Jhashank	Sumit

Architecture Block	Main Owner	Secondary Support
rPPG signal extraction	Jhashank	Aswin
rPPG feature engineering	Jhashank	Aswin
Quantum input conditioning	Jhashank	Aswin
Quantum classifier	Jhashank	Aswin
Output decision layer	Aswin	Jhashank
Evaluation	Aswin	Jhashank
Integration	Aswin	Sumit and Jhashank

Practical Note

This split is balanced because Sumit owns the full **data and face pipeline**, Jhashank owns the full **signal and model pipeline**, and Aswin owns the full **preprocessing policy, experiment control, evaluation, and integration pipeline** [web:255][web:261]. All three roles are implementation roles, and all three can proceed in parallel as long as agreed input-output formats are fixed at the start [web:258][web:259].